HACKING

ACTIVE DIRECTORY

Hands-On Attacks · Credential Abuse · Full Domain Compromise

J. Manassa Sathvika

3rd Year, B.E CSE (Cyber Security)

2025

Active Directory Attack Fundamentals — A Hands-On Learning Journey

Active Directory (AD) is Microsoft's directory service used to manage and organize users, computers, groups, and other resources within a Windows-based network. It provides a centralized system for authentication, authorization, and policy enforcement, ensuring that the right users have the right access to the right resources across an organization. In simple terms, AD acts as the central identity and access control hub for most enterprise networks.

To deepen my understanding of these concepts, I built a fully isolated Active Directory lab environment where I learned and practiced **beginner-level AD attack techniques**. Working hands-on helped me clearly see how attackers leverage weak configurations, credential exposure, and trust relationships to move through an AD environment.

This document outlines the lab environment I created and walks through the attack techniques I practiced, along with the key insights gained throughout the learning process. All activities were carried out safely in a sandboxed environment strictly for educational and skill-building purposes, with the goal of establishing a solid foundation in Active Directory security

My Lab Setup

This is the Active Directory lab environment I built and configured for practicing AD concepts and testing different attack scenarios.

Overview

- Small, isolated Active Directory lab built for hands-on learning and safe testing.
- Runs entirely on the host in a NAT/host-only virtual network so VMs can communicate without exposing services to the internet.

Virtual machines (4 total)

1. **Domain Controller — Windows Server 2019 – (Nimai-DC)**
    - Hosts Active Directory Domain Services (AD DS) and DNS for the lab domain (madhav.local).
    - Provides domain authentication, stores user/group objects and OUs, and hosts test file shares used for enumeration.
    - Role examples: domain administration, GPO management, service account/SPN registration.
2. **Client 1 — Windows 10 Enterprise (The-Supreme)**
    - Domain-joined user workstation representing a standard employee machine.
    - Local user: fcastle (Frank Castle).
    - Configured with a local administrator relationship for certain attack paths and a user file share to emulate typical user resources.
3. **Client 2 — Windows 10 Enterprise (The Spider-Man)**
    - Second domain-joined workstation to enable two-machine/lateral movement scenarios.
    - Local user: pparker (Peter Parker).

- Mirrored configuration of Client 1; used for attacks that require multiple targets (lateral movement, remote admin abuse).

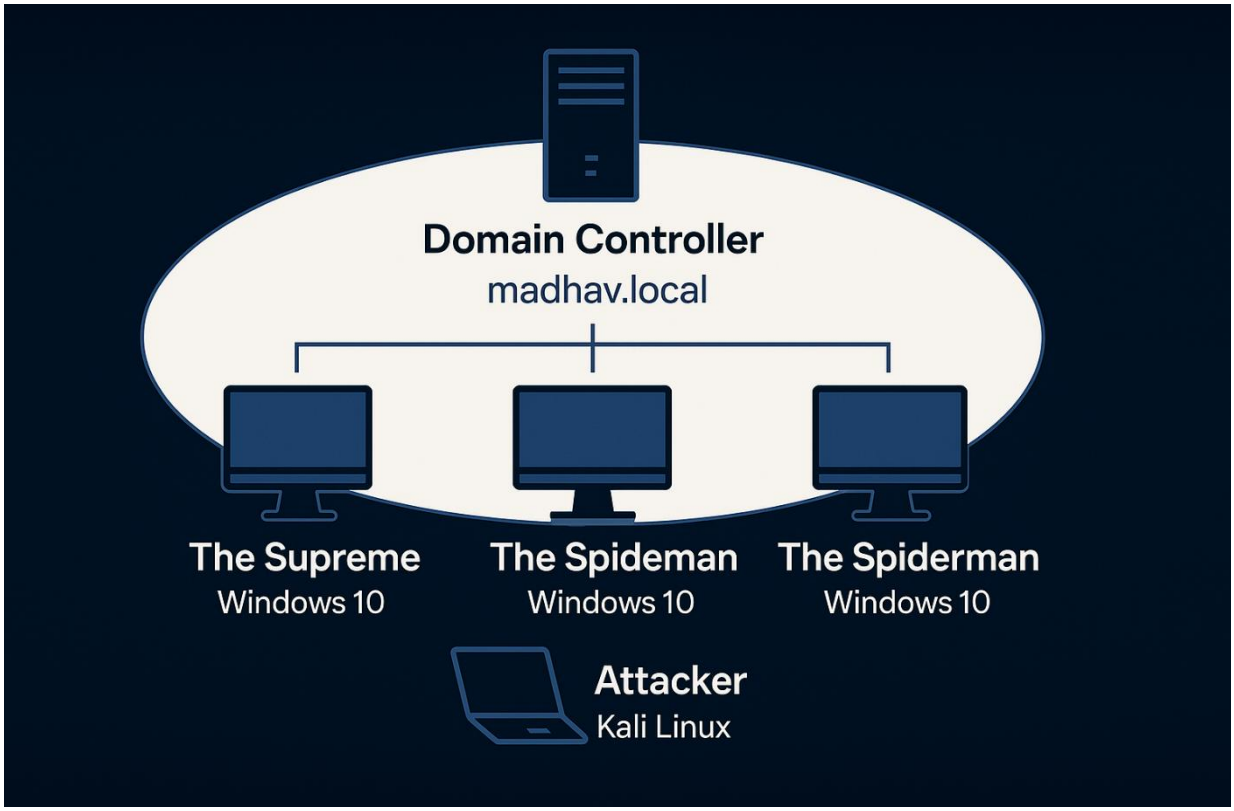
4. **Attacker — Kali Linux (offensive workstation)**
    - Used for reconnaissance, authentication attacks, Kerberos testing, SMB enumeration, and exploitation attempts against the lab.
    - Operates from the isolated network and targets the domain controller and clients.

Important accounts and roles

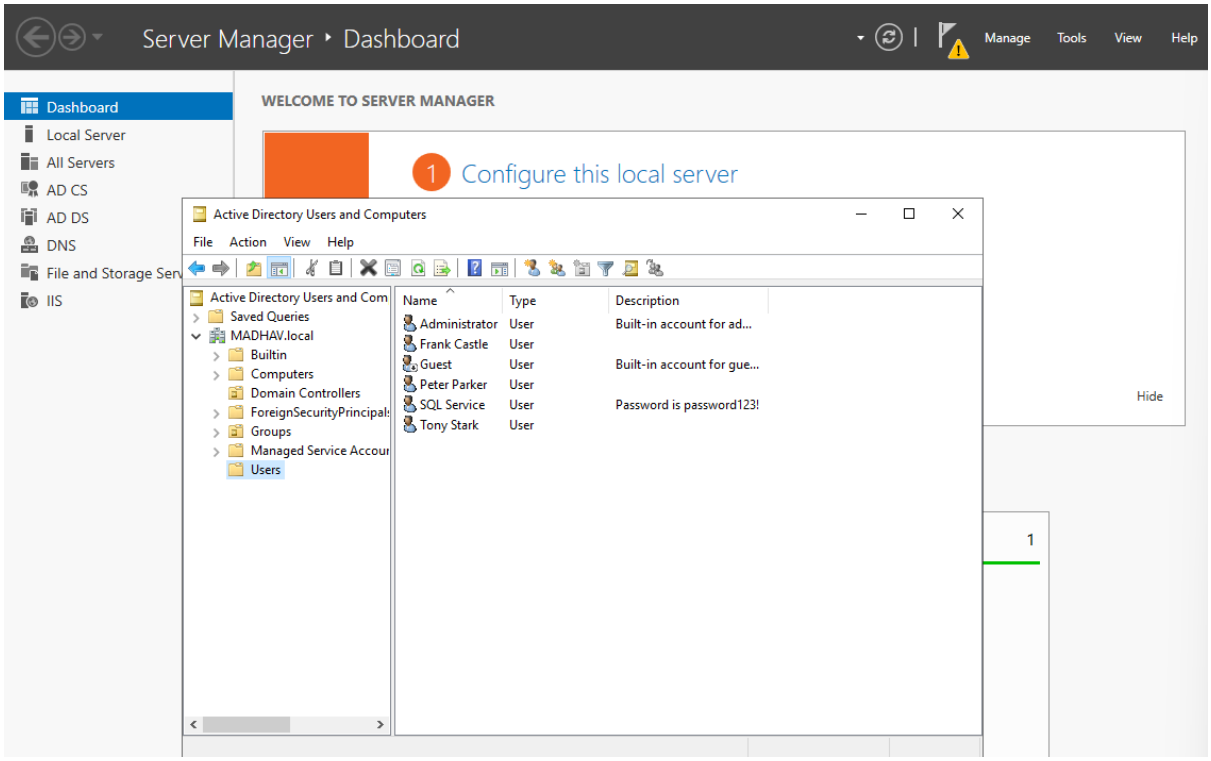
- **Domain Administrator(s)** — privileged accounts (example: tstark) with membership in Domain Admins used to demonstrate full-domain privilege scenarios.
- **Standard Domain Users** — lower-privilege accounts (examples: fcastle, pparker) used for normal login/privilege escalation exercises.
- **Service Account** — dedicated account for an application/SQL service (example: sqlservice) that is given a service principal name (SPN); used to demonstrate Kerberos service-ticket targeting and attacks against elevated service accounts.
- **Local Administrator relationships** — at least one user (fcastle) is configured as a local administrator on multiple client machines to enable realistic lateral movement techniques.

Key configurations (lab choices to enable learning)

- Domain named in a private namespace (e.g., madhav.local) to avoid public conflicts.
- DNS provided by the domain controller so Kerberos name resolution behaves like a production environment.
- File shares enabled on the domain controller and user workstations (SMB/ports opened) to simulate common resources.
- SPN registered for the service account to allow Kerberos service-ticket related exercises.



Virtual Active Directory lab architecture



Domain accounts and structure within MADHAV.local.

With the lab fully configured, I began testing a wide range of Active Directory attack techniques to understand how real networks can be compromised through authentication weaknesses and misconfigurations. I started with LLMNR/NBT-NS poisoning to capture and crack NTLMv2 hashes, followed by SMB relaying to reuse those credentials for unauthorized access. Using Impacket tools, I established remote shells on domain hosts, enabling deeper interaction and setup for more advanced attacks. I then performed IPv6 DNS hijacking with mitm6, which allowed me to manipulate authentication flows and escalate privileges through LDAP relay attacks. Building on this access, I carried out Pass-the-Password and Pass-the-Hash techniques for lateral movement, and used token impersonation to elevate privileges through cached authentication tokens.

Next, I executed Kerberoasting to extract and crack weak service account passwords, and used a URL-based credential harvesting trick to capture additional NTLMv2 hashes from shared folders. After gaining control of the Domain Controller, I ran Mimikatz to dump plaintext credentials, NTLM hashes, LSA secrets, and Kerberos data. This led to the final step: generating a Golden Ticket, providing unrestricted, persistent access across the entire domain. Together, these attack chains demonstrated how a single weak point can escalate into full domain compromise when multiple techniques are combined.

LLMNR Poisoning

Objective:
To capture a user's NTLMv2 authentication response and ultimately recover the user's password by impersonating a network resource during failed hostname resolution.

Concept:
When a Windows system cannot resolve a hostname through DNS, it falls back to LLMNR and NetBIOS name resolution. These protocols broadcast queries across the local network. An attacker can intercept these broadcasts, pretend to be the requested resource, and cause the victim machine to send its NTLMv2 authentication response.

Tools Used:
Responder (for capturing NTLMv2 responses)

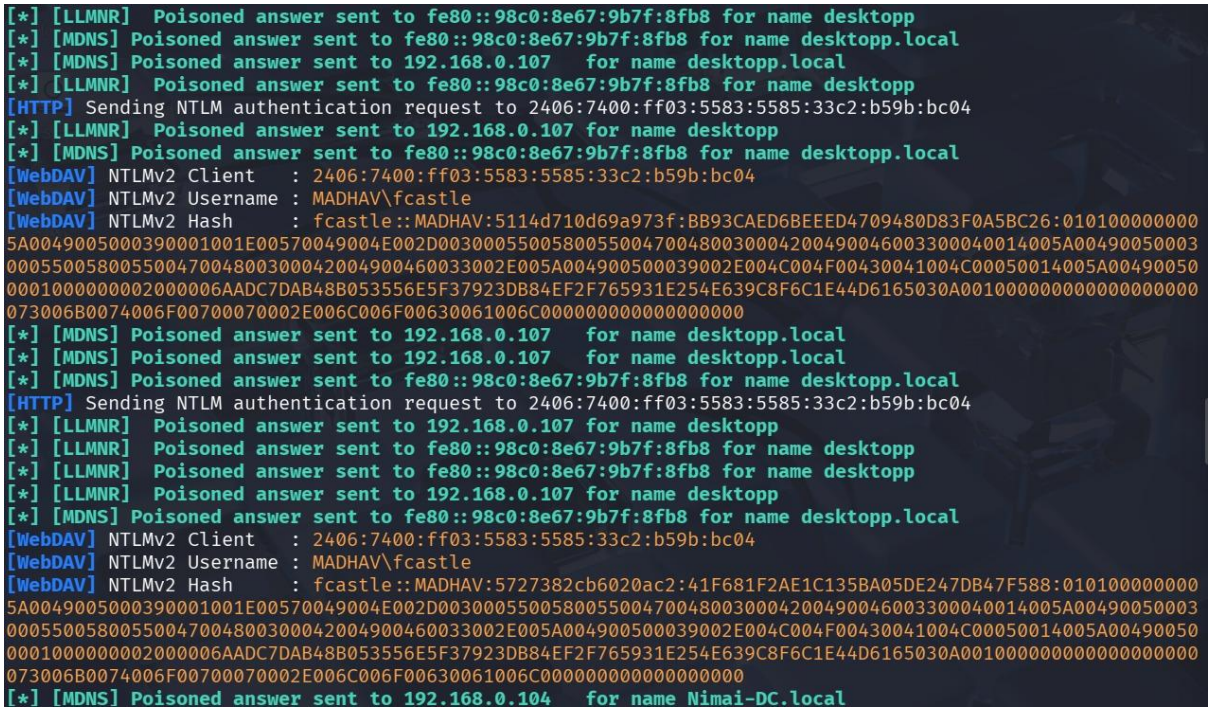
Hashcat (for offline password cracking).

What I Did in the Lab:
I activated a responder tool on the attacker machine to listen for LLMNR and NetBIOS broadcasts. By triggering a failed resource lookup from a domain-joined client, the system broadcasted an LLMNR request. The attacker machine intercepted this request and captured the NTLMv2 response belonging to the user **fcastle**.

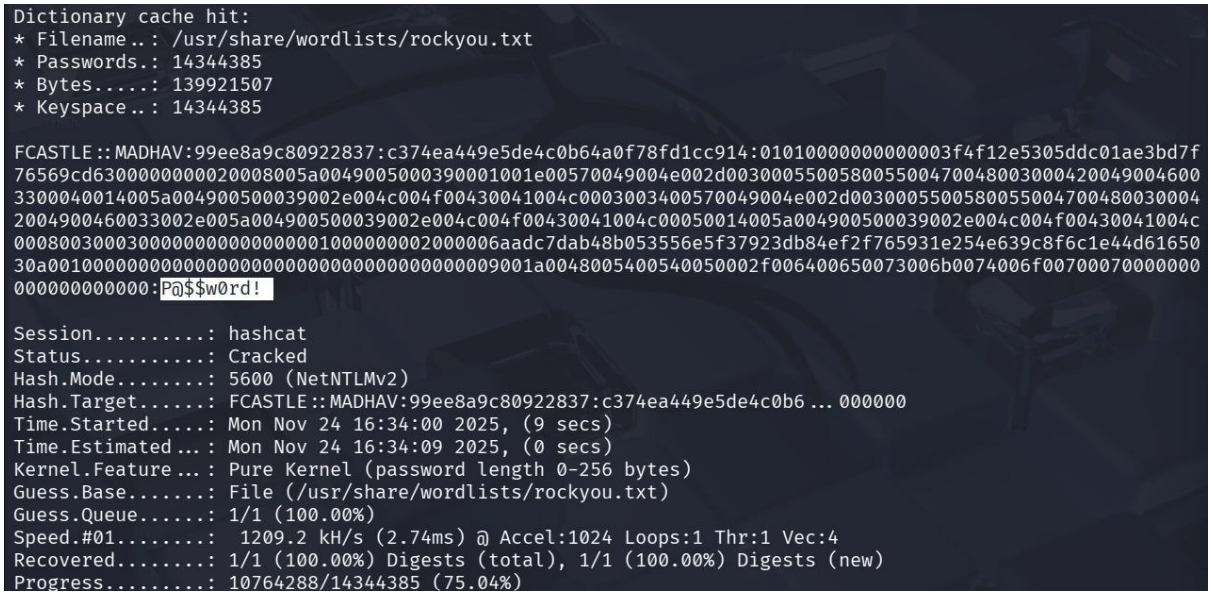
Outcome:
The captured NTLMv2 hash was successfully cracked with a wordlist, revealing the user's weak password.

Mitigation:

- Disable LLMNR and NetBIOS across endpoints
- Enforce strong password policies
- monitor for suspicious responder-like activity on internal networks



NTLMv2 hash captured via LLMNR poisoning.



Successful password recovery from captured NTLMv2 hash.

SMB Relay

Objective:
Use captured authentication material to authenticate to another host and gain elevated access (dump local SAM hashes or obtain an interactive shell).

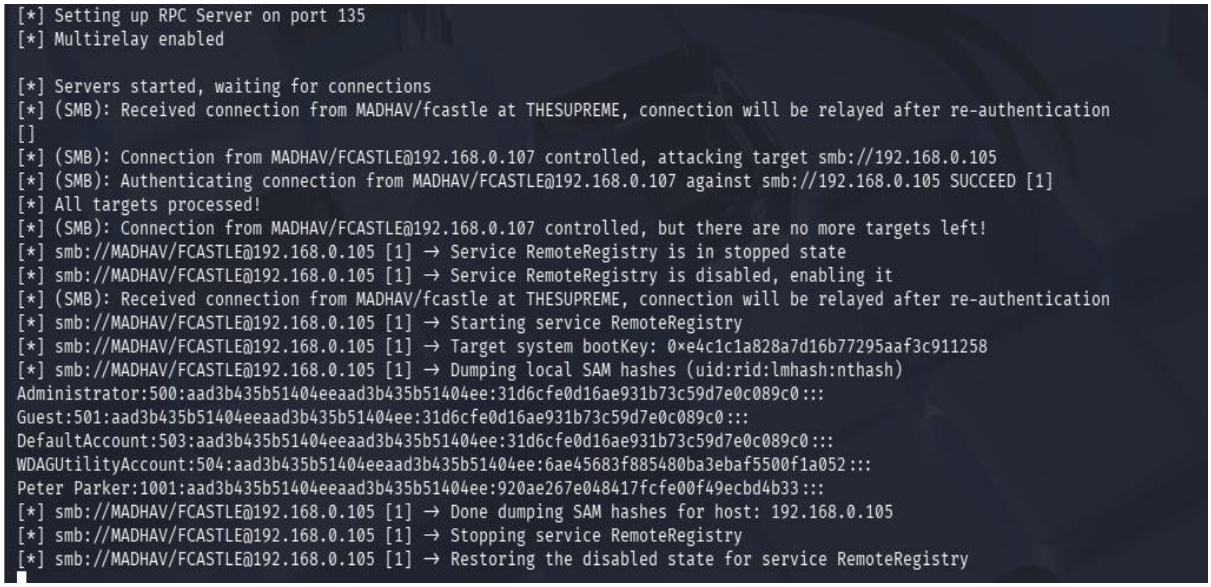
Concept:
SMB relay takes an intercepted NTLM authentication exchange and forwards (relays) it to a different target host. If the target allows NTLM authentication and SMB signing is not required, the relayed credentials can authenticate successfully. This is especially effective when the relayed account has local administrator privileges on the target machine.

Tools used:
Responder (to capture credentials) and ntlmrelayx (to relay captured credentials to target hosts and, optionally, spawn an interactive SMB shell).

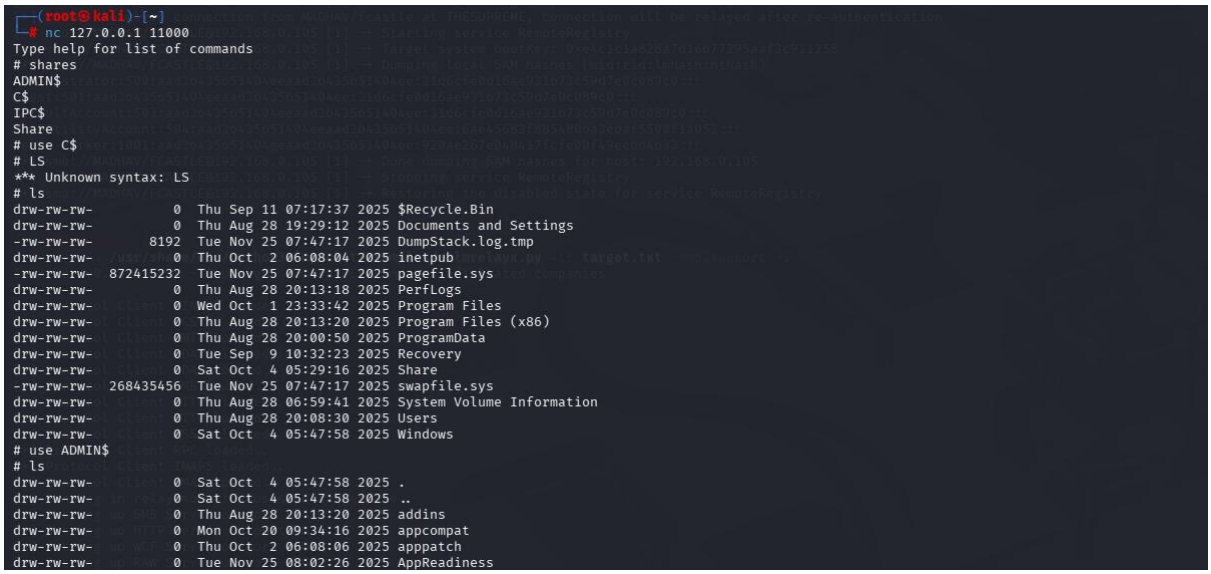
What I did in the lab:
I configured Responder to capture broadcast authentication attempts while disabling its SMB/HTTP responders, then used ntlmrelayx with a targets list to relay captured credentials from one workstation to a second workstation where the same user was a local administrator. After a successful relay, I dumped the target's local SAM data and, in a follow-up run, obtained an interactive SMB shell.

Outcome:
The relay succeeded because SMB signing was not required on the target and the relayed account had local admin rights. This allowed dumping of local user hashes (SAM) and the creation of an interactive shell on the target machine.

Mitigation:
Enable SMB message signing or require signing on all endpoints, disable NTLM authentication where feasible, apply account tiering (limit high-privilege accounts from logging into low-tier systems), and restrict local administrator membership to minimize lateral movement risk.



Relayed NTLM authentication allowing access and SAM dump



Interactive SMB shell obtained on the target via ntlmrelayx relay.

Gaining Shell Access

Objective:
To use previously obtained credentials to authenticate remotely to a target Windows machine and gain a functional shell for post-exploitation.

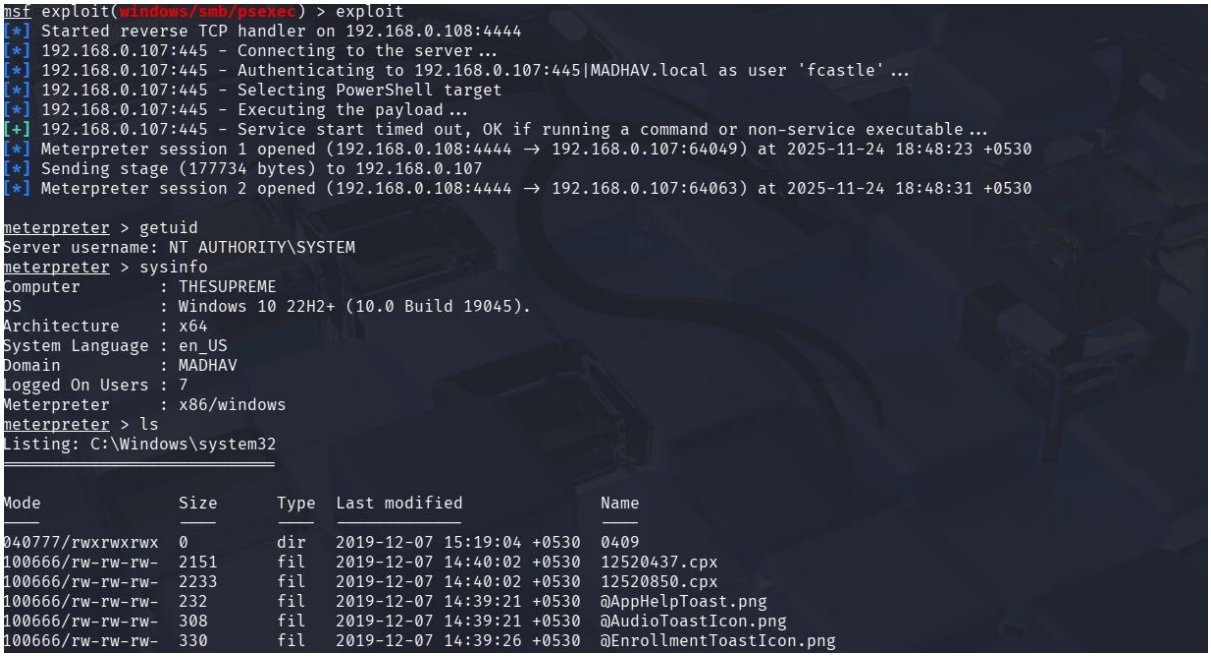
Concept:
When a valid user credential is available and that user has local administrator rights on a machine remote execution technique such as PSExec, SMBExec, or WMIExec can be used to establish a shell. These methods rely on SMB/WMI access and are often detected by antivirus solutions, so attackers typically try multiple execution paths.

Tools Used:
Metasploit PSExec module, psexec.py, smbexec.py, and wmiexec.py (Impacket toolkit).

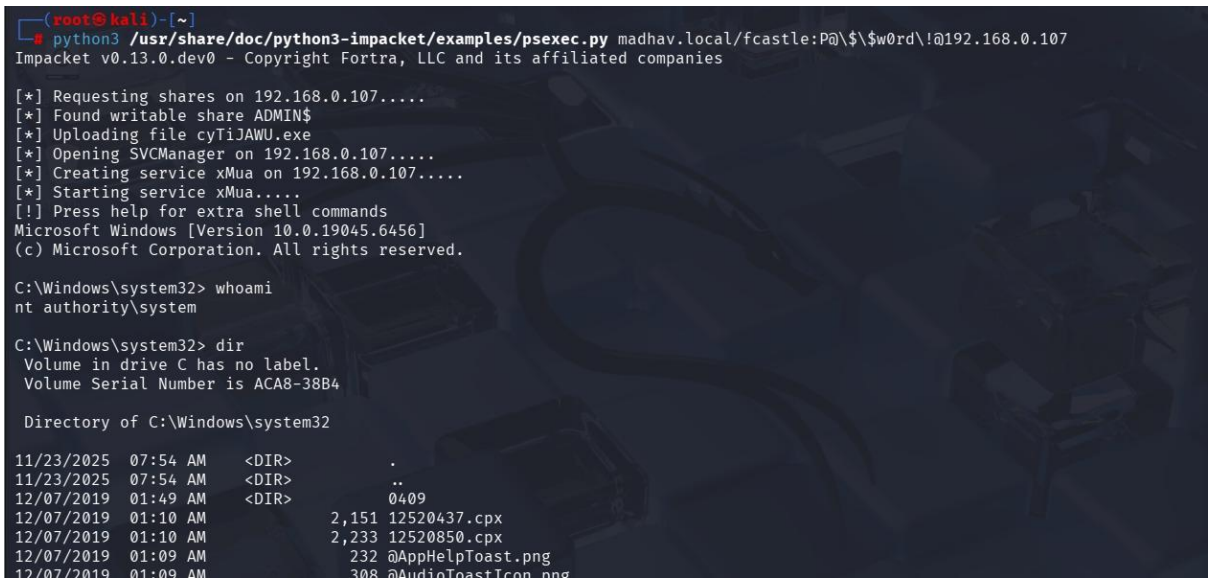
What I Did in the Lab:
Using the valid credential recovered from the LLMNR Poisoning attack, I attempted remote authentication against a workstation where the same user (fcastle) had local administrator privileges. After Metasploit's PSExec was blocked by Windows Defender, I switched to Impacket tools. psexec.py successfully established a remote shell, and additional testing with SMBExec and WMIExec showed alternative command-execution paths.

Outcome:
A remote shell was successfully obtained on the target system using psexec.py, confirming that a cracked credential combined with local admin rights provides full remote access.

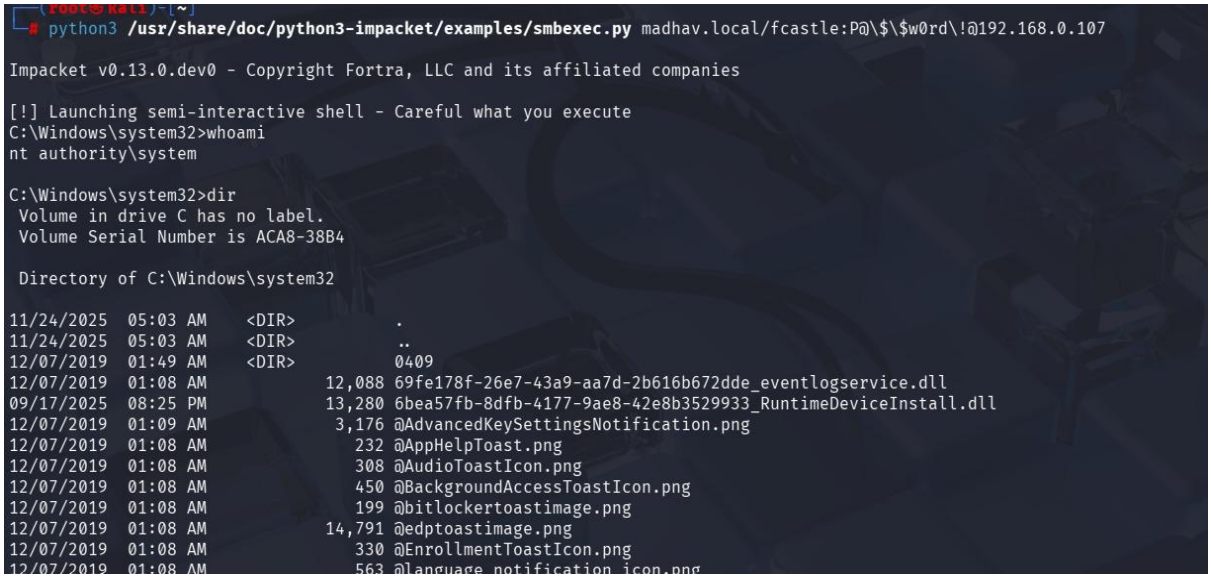
Mitigation:
Restrict local administrator memberships, enforce account-tiering practices, monitor for remote execution activity, and strengthen endpoint defenses to detect/block PSExec/WMI-style behavior.



Meterpreter shell obtained on the target system using valid credentials.



SYSTEM-level command shell gained via Impacket psexec.py.



Semi-interactive SYSTEM shell established using smbexec.py.

IPv6-based Man-in-the-Middle / DNS Relay

Objective:
To intercept IPv6 name-resolution traffic, impersonate IPv6 DNS responses, and capture or relay authentication material to gain privileged access or create persistence in the domain.

Concept:
Many Windows hosts have IPv6 enabled even when the network primarily uses IPv4, and IPv6 DNS resolution is often unattended. By advertising malicious IPv6 DNS responses, an attacker can redirect a host's IPv6 traffic (including SMB/LDAP) to an attacker-controlled machine. Captured NTLM exchanges can then be relayed (for example via LDAP relay) to the domain controller or other targets, allowing account creation, privilege escalation, or remote execution if the relayed principal has sufficient rights.

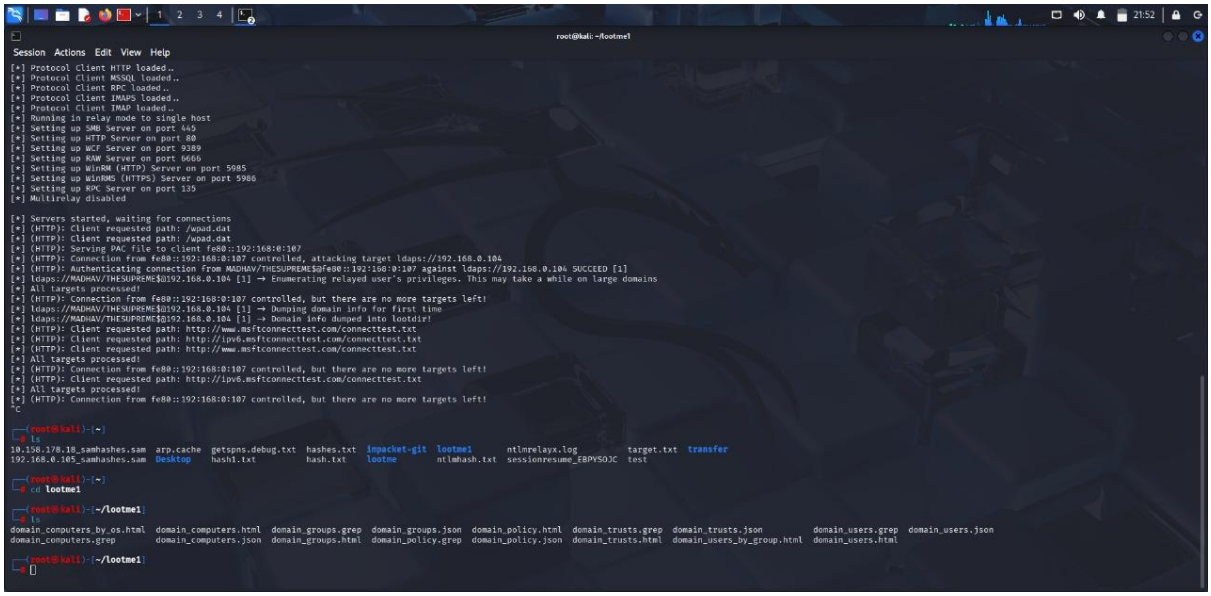
Tools used:
mitm6 (IPv6 MITM/DNS spoofer) and ntlmrelayx (for relaying captured NTLM authentication to LDAP/SMB targets). Certificate support (LDAPS) is used where required to enable LDAP relaying.

What I did in the lab:
I installed and ran mitm6 on the attacker host to respond to IPv6 DNS queries on the isolated lab subnet. I ensured LDAPS was available on the domain controller (installed a lab CA/ certificate) so relayed authentication could target LDAP over TLS. When a lab workstation rebooted (triggering IPv6 resolution), mitm6 redirected its IPv6 traffic to the attacker. Using ntlmrelayx I relayed captured NTLM material to the domain controller, which allowed the creation of an account and extraction of domain information via the relayed session.

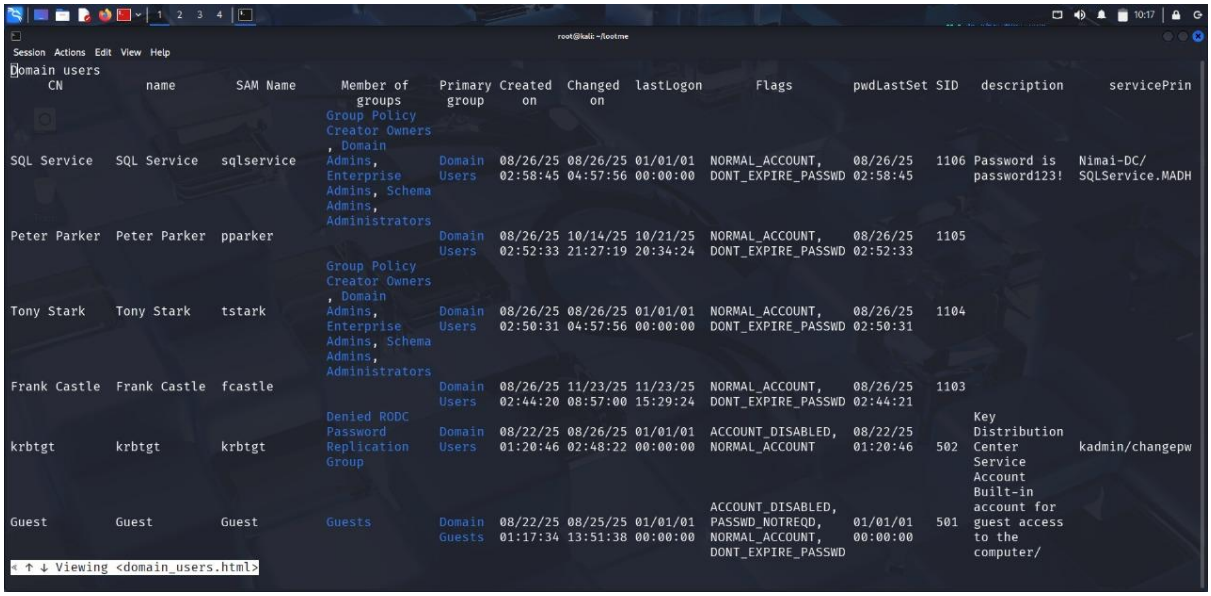
Outcome:
The attack successfully intercepted IPv6 DNS traffic and relayed NTLM authentication to the domain controller. This allowed the extraction of valuable domain information (users, groups, policies, trusts) and enabled the automatic creation of a new domain account with a modified ACL that granted elevated access.

Mitigation:

- Disable unused IPv6 services or enforce strict IPv6 firewall rules (deny by default / allow necessary).
- Disable WPAD or block WPAD discovery if not required.
- Enable LDAP signing and channel binding (LDAPS) to reduce relaying success.
- Use the Protected Users group and account tiering to prevent high-privilege credentials from being usable on low-tier systems.
- Monitor for anomalous IPv6 DNS advertisements and unusual LDAP/SMB authentication activity.



ntlmrelayx capturing relayed authentication and dumping domain information



Domain user listing recovered through LDAP relay using mitm6.

Post Compromise attack

Pass-the-Password & Pass-the-Hash

Objective:
Leverage already-compromised credentials (clear-text passwords and NTLM hashes) to move laterally across the network and gain additional access without needing to re-crack accounts.

Concept:
After initial compromise, an attacker often obtains either:

- **Passwords** (e.g., from LLMNR poisoning / cracked hashes), or
- **NTLM hashes** (e.g., from SAM/LSA dumps).

Instead of only using these on a single host, the attacker can "spray" them against many machines:

- **Pass-the-password:** directly authenticate using a known username/password combination on multiple hosts.
- **Pass-the-hash:** authenticate using the NTLM hash in place of the actual password, without knowing the clear text.

These techniques are powerful when local administrator accounts are reused across many systems.

Tools Used:

- crackmapexec – to test credentials and hashes across the subnet (pass-the-password and pass-the-hash).
- secretsdump.py (Impacket) – to dump local SAM/NTLM hashes from compromised hosts.
- hashcat – to crack NTLM hashes where possible.
- psexec.py (Impacket) – to obtain shells using valid credentials or hashes.

What I Did in the Lab:
Using the **fcastle/p@$$w0rd!** credentials (originally obtained from earlier attacks), I ran crackmapexec against the lab subnet to see where that account

worked. This confirmed that fcastle was a local administrator on both **The Supreme** and **The Spiderman**.

Next, I used secretsdump.py from those machines to extract local NTLM hashes, I then:

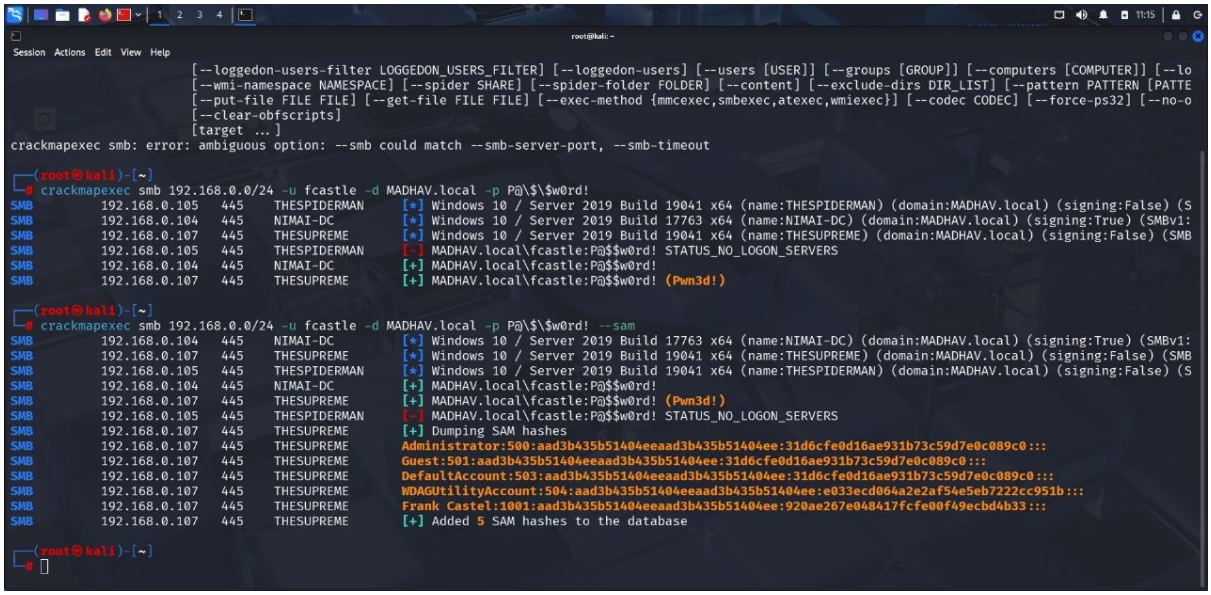
- **Passed the password** with crackmapexec to identify additional reachable systems.
- **Passed the hash** (using crackmapexec and psexec.py) to test whether the local admin hashes worked on other hosts and to attempt remote shells, all without needing the clear-text password each time.

Outcome:

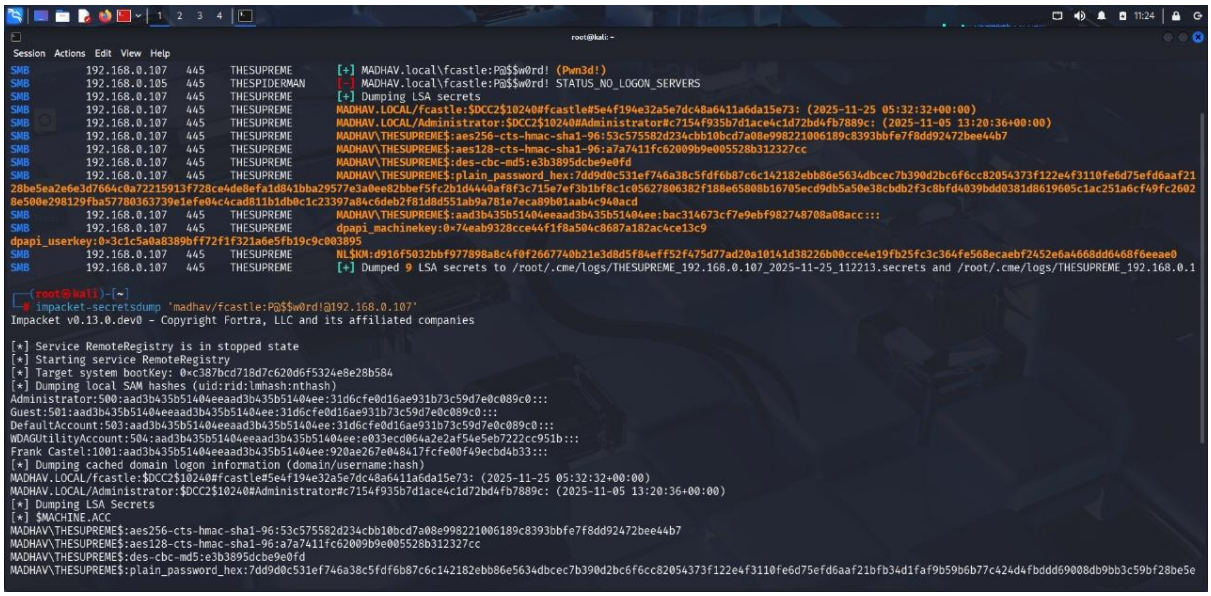
- Passing the password confirmed that the same local admin credential was reused on multiple machines, giving me administrative access beyond the initial host.
- Secretsdump provided local NTLM hashes, some of which were cracked, giving more valid credentials and insight into the organization's weak password practices.
- Pass-the-hash attempts showed that these local admin hashes could be reused for authentication, demonstrating how a single compromised local account can quickly lead to control over multiple systems.

Mitigation:

- Avoid reusing local administrator accounts and passwords across machines; use unique local admin credentials or solutions like LAPS/PAM.
- Enforce strong, complex passwords and good password hygiene to reduce cracking success.
- Apply least privilege: minimize who is a local administrator and on how many systems.



CrackMapExec Pass-the-Password scan showing successful authentication and system compromise on multiple hosts.



CrackMapExec SAM dump revealing local account password hashes from the compromised machine.

Token Impersonation Attack

Objective

To escalate privileges by impersonating an authenticated user's security token potentially obtaining Domain Admin rights without knowing their password.

Concept

Windows assigns each logged-in user a **token**, similar to a session cookie.
If an attacker compromises a machine and finds a token belonging to a privileged user, they can **impersonate** that identity using Meterpreter's **incognito** module.

Two token types matter:

- **Delegate Tokens:** Created during interactive logins (RDP, console).
- **Impersonation Tokens:** Created when accessing shared network resources.

If a Domain Admin has logged into a workstation and a token is still present, the attacker can immediately act as that Domain Admin.

Tools Used

- Metasploit (PSExec)
- Meterpreter with the incognito extension

What I Did in the Lab

I gained a Meterpreter session on Frank Castle's machine, loaded **incognito**, and listed available tokens.
Initially only Administrator's token was present. After logging Frank Castle into the machine, his delegate token appeared too.

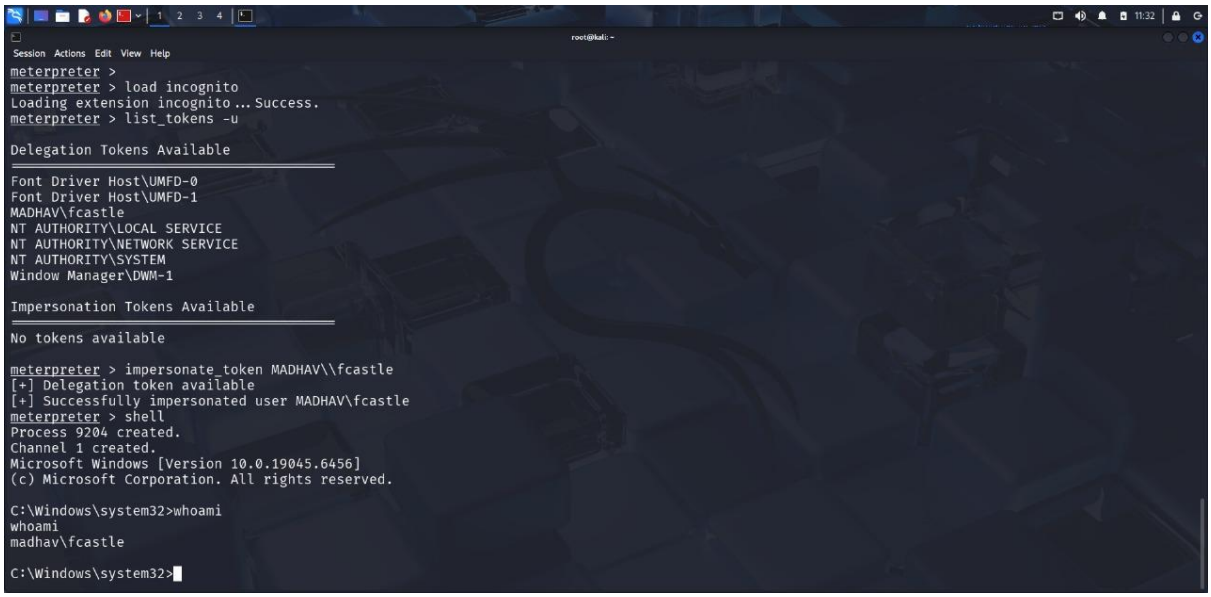
Using impersonate_token, I switched identities (Administrator → Frank Castle) without knowing either password.
Once impersonated, I verified access using basic shell commands and could perform privileged actions.

Outcome

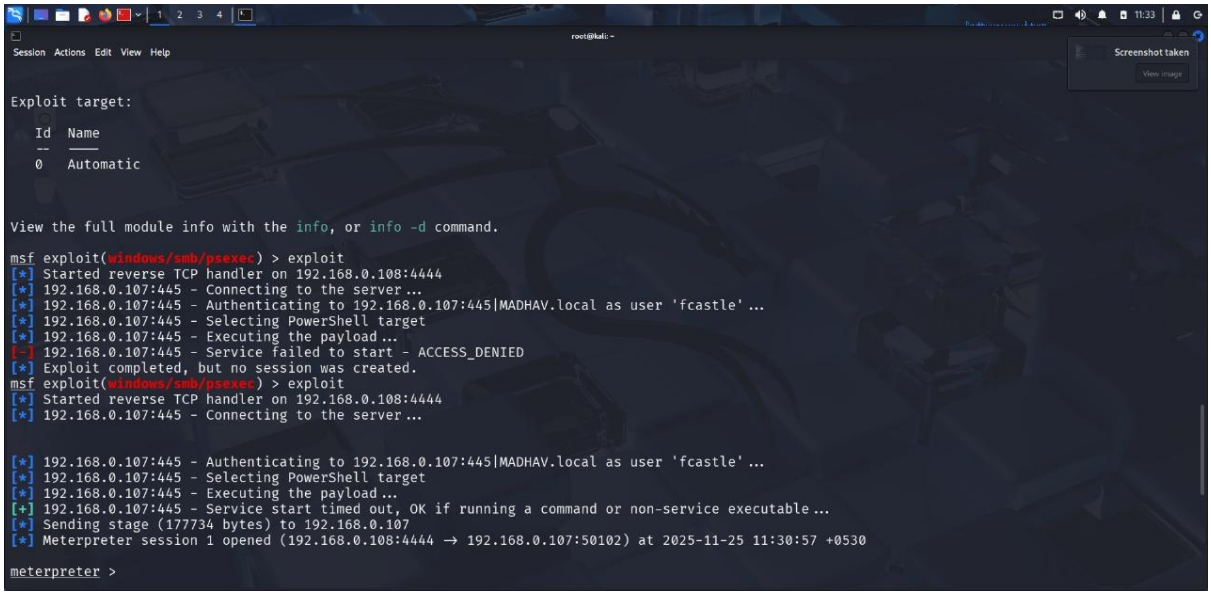
- Successfully impersonated multiple users based on available tokens.
- Demonstrated how a single logged-in Domain Admin token on a workstation would immediately give **full domain compromise**.

Mitigation

- Avoid logging Domain Admin accounts into workstations; use strict **account tiering**.
- Remove unnecessary **local admin rights** from users.
- Reboot systems regularly to clear stale tokens.
- Use separate privileged accounts (e.g., user vs user-admin).



Listing and impersonating available user tokens using Incognito inside Meterpreter.



Gaining a Meterpreter session on the target system via PSExec after successful authentication.

Kerberoasting

Objective

Obtain service account passwords by requesting and cracking Kerberos service tickets with the goal of escalating privileges and potentially compromising the entire domain.

Concept

Kerberos allows any valid domain user to request Service Tickets (TGS) for accounts running configured services (SPNs).
These tickets are encrypted using the service account's NTLM hash.
If the service account has a weak password, the encrypted ticket can be cracked offline, revealing the plaintext password.
Misconfigured environments often run services with high privilege accounts, making this attack extremely valuable.

What I Did

Using the compromised domain credentials for *fcastle*, I executed Impacket's GetUserSPNs.py to enumerate service principal names and request the associated Kerberos service tickets. This allowed me to obtain a TGS ticket for the SQL service account. I then extracted the returned Kerberos hash and loaded it into Hashcat (mode 13100) to perform offline cracking. The hash quickly resolved to a weak, guessable password, confirming that the environment had poorly secured service account credentials.

Outcome

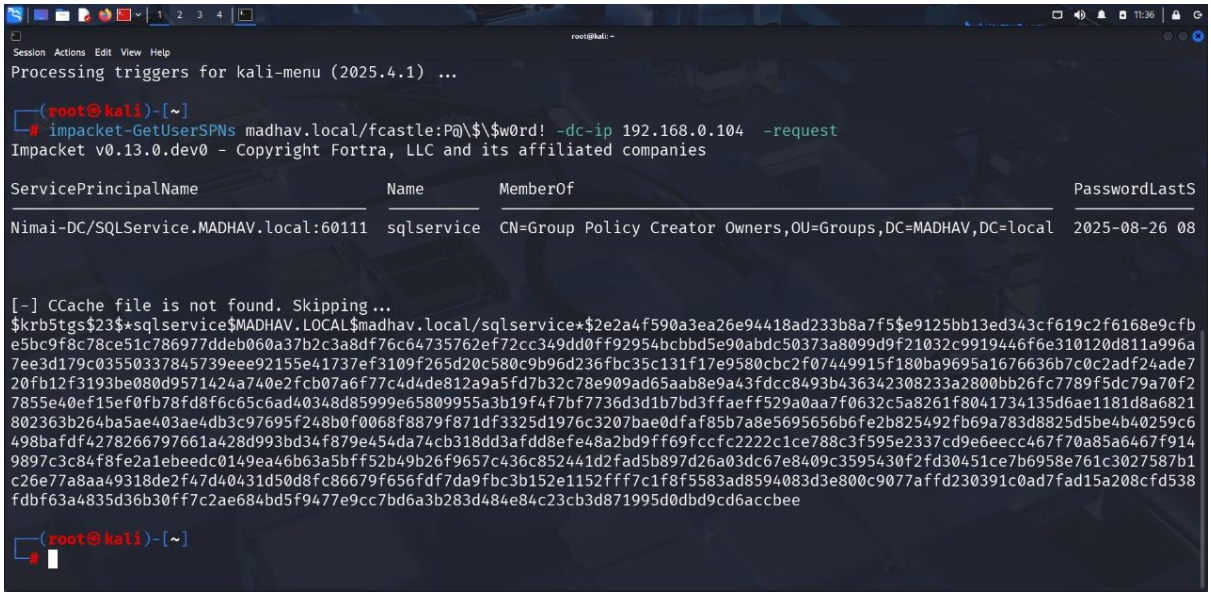
The attack successfully recovered the plaintext password for the SQL service account, which was both weak and improperly configured with elevated privileges. Because the service account had been granted Domain Admin rights, cracking its password effectively provided a direct path to full domain compromise. This demonstrated a critical misconfiguration where a high-privilege service account was protected by a trivial password, enabling rapid privilege escalation through Kerberoasting.

Tools Used

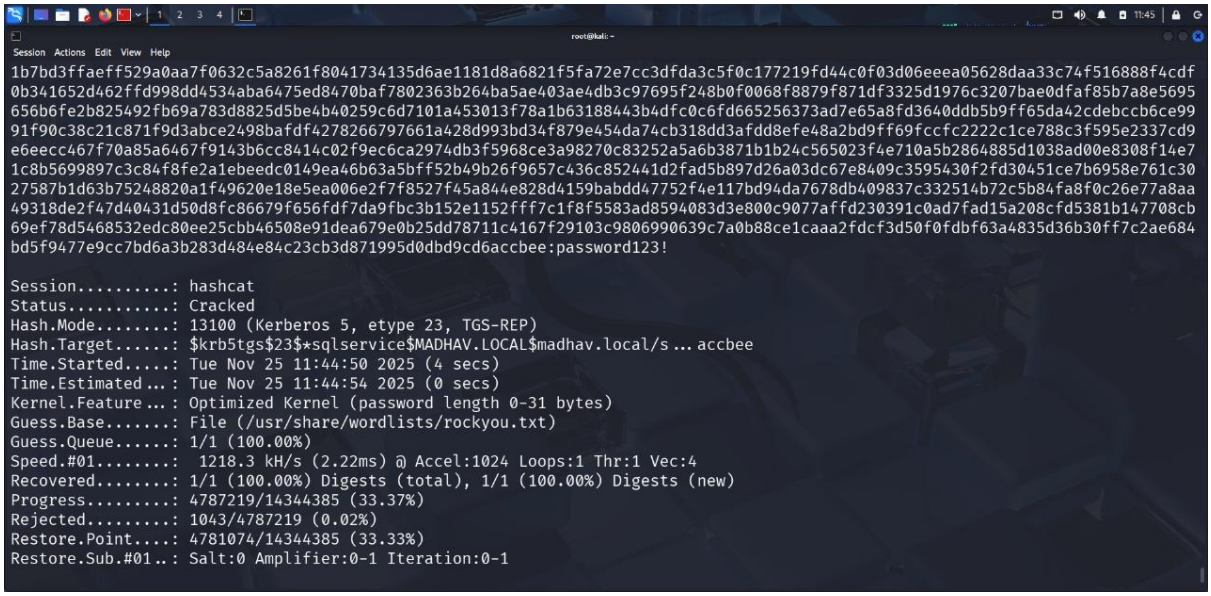
- GetUserSPNs.py (Impacket) — to request TGS tickets
- Hashcat — to crack the Kerberos ticket hash

Mitigation

- Configure strong, long, random passwords for all service accounts (25+ characters).
- Avoid assigning Domain Admin rights to service accounts.
- Apply strict least privilege principles and review SPN assignments regularly.
- Rotate service account passwords on a scheduled basis.



Kerberoasting request sent — service ticket (TGS) successfully extracted for SQLService.



Hashcat cracked the Kerberoasted service hash, revealing the SQLService password: password123!.

URL File Attack

Objective

To leverage a compromised user account or an accessible network file share to capture NTLMv2 hashes by forcing the victim's system to authenticate to the attacker-controlled machine.

Concept

A URL file attack abuses Windows behaviour where .url (Internet Shortcut) files automatically attempt to resolve their target path.
By placing a crafted .url file inside a shared folder, any user who merely opens the folder triggers an outbound authentication request.
This request is sent to the attacker's IP, where Responder captures the NTLMv2 hash.
This hash can then be cracked or relayed for further lateral movement.

Tools Used

- Responder – to capture NTLMv2 hashes
- Any writable SMB share (compromised user or open share)
- Notepad – to craft the malicious .url file

What I Did in the Lab

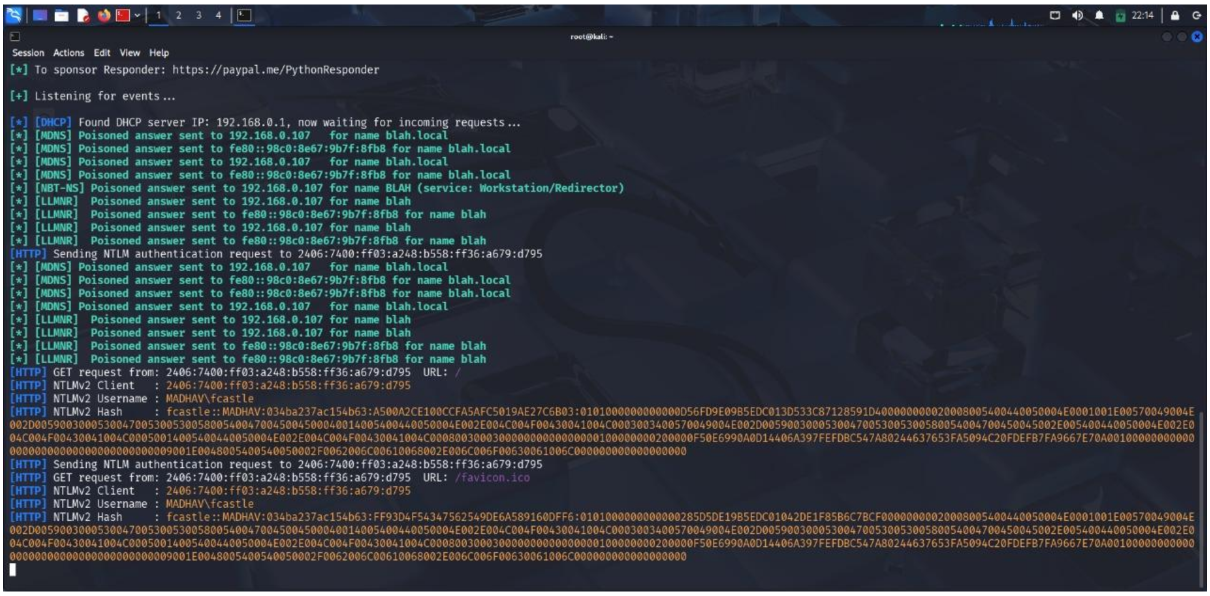
I first created a malicious @test.url file containing a URL field that pointed directly to my Kali machine's IP address. Then, using the compromised user *Frank Castle*, I placed this .url file inside the HackMe shared folder on the domain controller. Next, I launched Responder in verbose mode to listen for incoming authentication attempts. As soon as the user browsed into the shared folder, the .url file automatically tried to reach the attacker IP, causing Windows to send an NTLMv2 authentication request. Responder captured this hash instantly, giving me a new credential that could potentially be cracked or relayed to escalate privileges.

Outcome

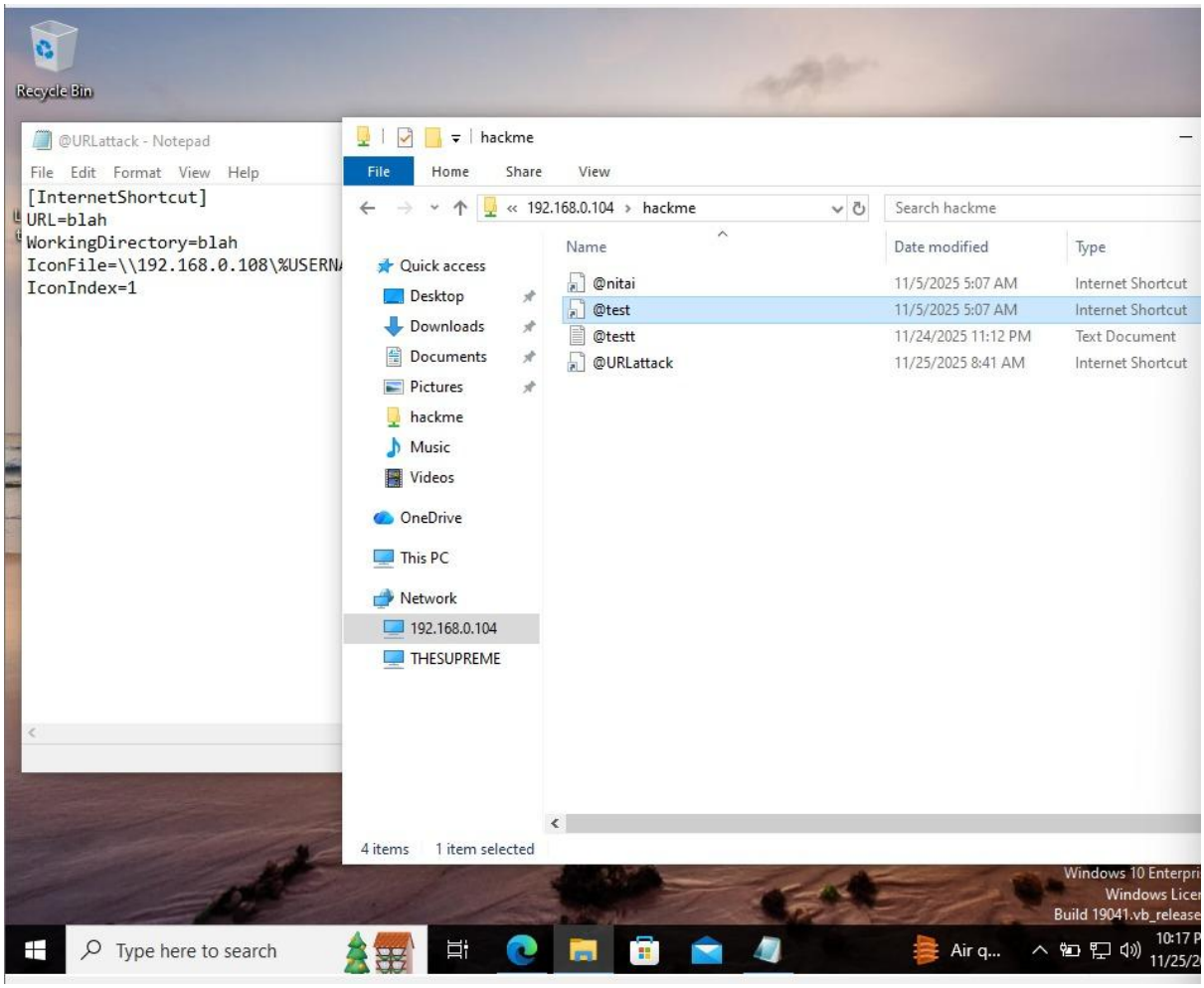
A valid NTLMv2 hash was successfully captured just by having the victim open the shared folder. No user interaction beyond navigating to the directory was required, making this a highly reliable and stealthy credential-harvesting technique.

Mitigation

- Block LLMNR/NetBIOS name resolution (Responder relies on this).
- Use SMB signing and NTLM relay protections.
- Enforce strong passwords and MFA to reduce hash-cracking success.
- Limit write access to shared folders to prevent attackers from placing malicious files.



Responder capturing NTLMv2 hashes after the victim accesses the crafted URL file on the shared folder.



Malicious .url shortcut placed in the shared folder to trigger automatic authentication and leak hashes.

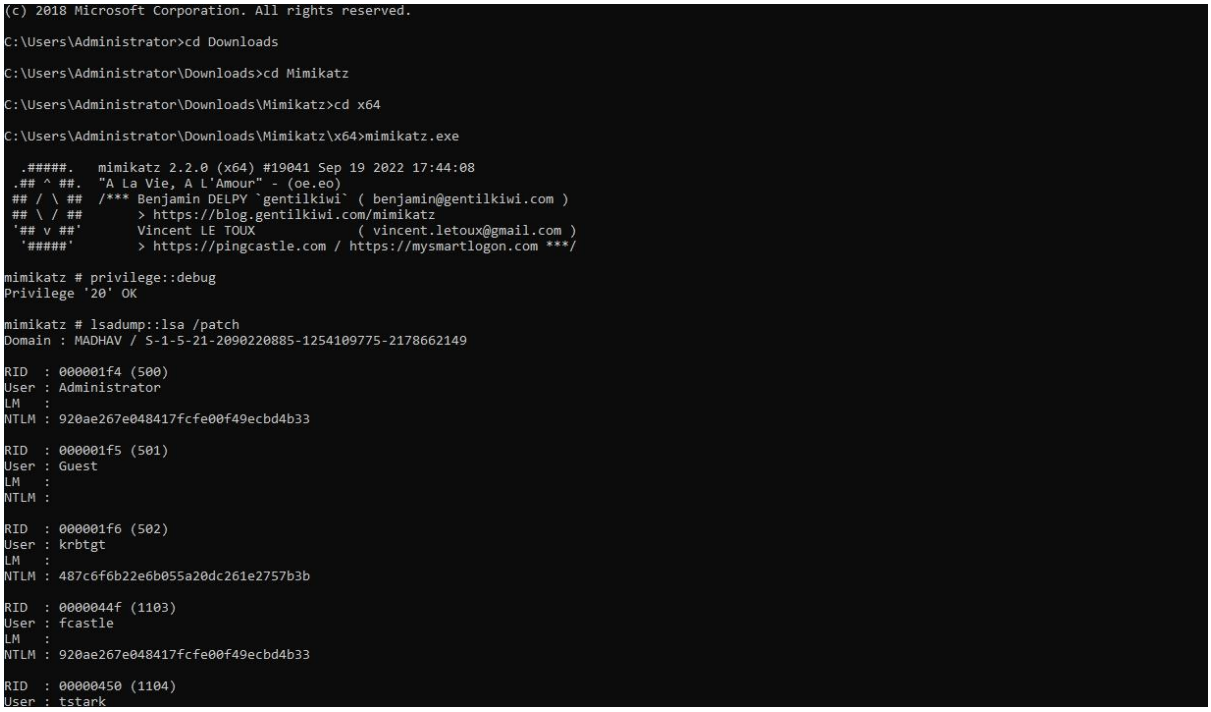
Mimikatz

What I Did

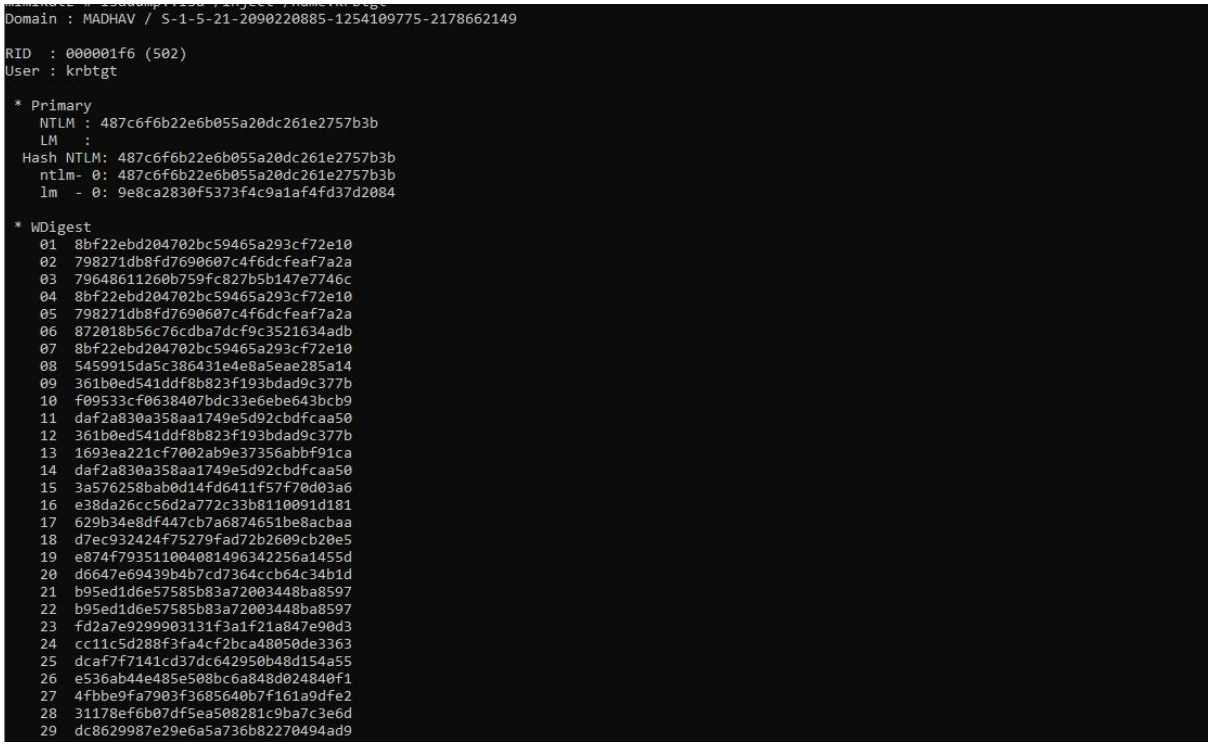
After gaining access to the Domain Controller and establishing a shell, I executed Mimikatz on the system. With full privileges on the DC, I enabled the necessary debug rights and used Mimikatz modules to dump credentials from memory. This included extracting logon passwords, NTLM hashes, LSA secrets, and Kerberos ticket information.

Outcome

Mimikatz successfully dumped NTLM hashes and plaintext credentials (where available) for both local and domain users. It also retrieved Kerberos-related data that can be used for advanced attacks such as Pass-the-Hash, Pass-the-Ticket, or crafting Golden Tickets. This demonstrated complete credential exposure once the Domain Controller was compromised.



Executing Mimikatz on the Domain Controller after privilege escalation, enabling full credential dumping via privilege::debug and lsa::dump



Dumped NTLM hashes and WDigest credentials, including the KRBTGT account—critical data for attacks like Golden Ticket creation.

Golden Ticket Attack

Objective

To forge a Kerberos Ticket-Granting Ticket (TGT) using the KRBTGT account hash and gain unrestricted, persistent access to all systems in the domain.

Concept

A Golden Ticket attack abuses the fact that the KRBTGT account is responsible for signing all Kerberos tickets.
If an attacker obtains the KRBTGT NTLM hash, they can generate their own TGTs for *any* user, at *any* privilege level, with *any* lifetime — effectively granting total domain compromise.

Because Kerberos trusts these tickets by design, they are accepted as legitimate across the entire domain.

Tools Used

- Mimikatz — to extract the KRBTGT hash and generate the forged Golden Ticket.
- cmd / PowerShell — to test domain access using the injected ticket.

What I Did in the Lab

- After gaining full access to the Domain Controller, I executed Mimikatz.
- Used privilege::debug to enable required permissions.
- Extracted the Domain SID and the KRBTGT NTLM hash using lsadump::lsa /patch.
- Generated a Golden Ticket using kerberos::golden, specifying:
    - User (Administrator)
    - Domain name
    - Domain SID
    - KRBTGT hash
    - RID 500

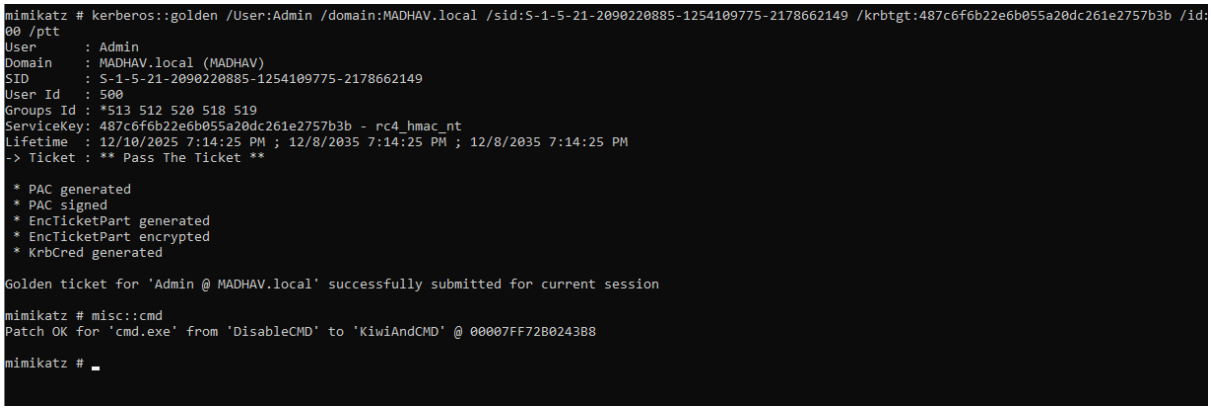
- Injected the Golden Ticket into the current session using /ptt.
- Opened a new shell and accessed remote systems and administrative shares without providing any credentials.

Outcome

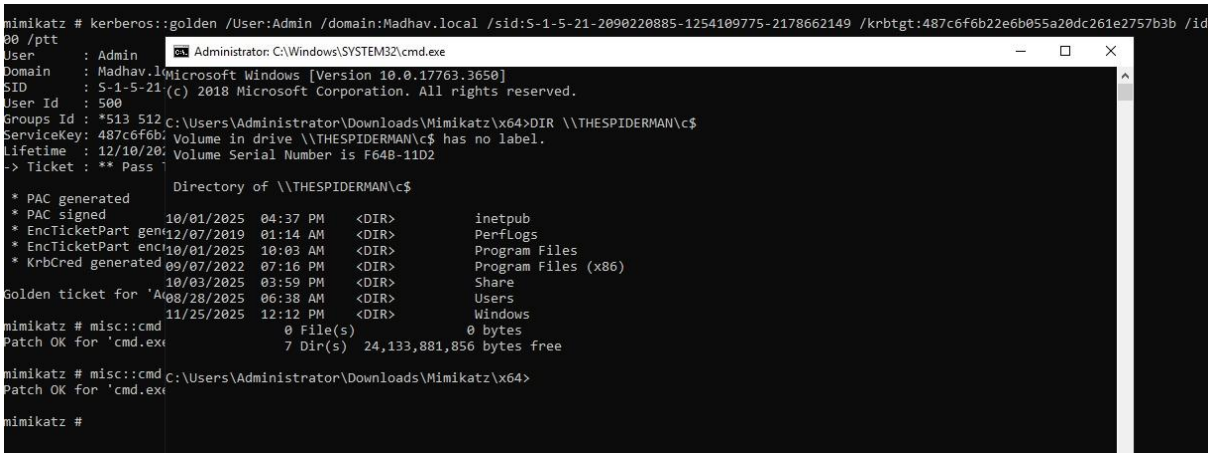
- Successfully generated and injected a Golden Ticket.
- Obtained complete, unrestricted domain access.
- Could browse shared drives (e.g., \\Punisher\C$) and authenticate to any domain machine as a Domain Admin.
- Demonstrated persistence, as Golden Tickets remain valid until the KRBTGT password is reset *twice*.
- Confirmed full domain compromise.

Mitigation

- Reset the KRBTGT password twice to invalidate all Golden Tickets.
- Implement Credential Guard, LSA Protection, and restrict DC access.
- Enforce tiered administrative model to prevent KRBTGT exposure.
- Regularly audit for abnormal Kerberos ticket lifetimes and unusual logon behavior.
- Limit who can log into the Domain Controller and monitor for unauthorized privilege escalation.



Golden Ticket successfully generated and injected using Mimikatz, granting a forged TGT for domain-wide authentication.



Using the injected Golden Ticket to access remote administrative shares (e.g., \THESPIDERMAN\C$) without credentials, confirming full domain compromise.

Conclusion

This Active Directory lab project gave me hands-on experience with the full attack lifecycle from initial credential capture to complete domain compromise. By practicing beginner-level techniques in a safe environment, I developed a clear understanding of how real-world attackers pivot through misconfigurations, weak authentication controls, and privilege escalation paths.

Working step-by-step through these attacks strengthened my foundational skills in AD exploitation and helped me understand the importance of secure configuration, monitoring, and credential hygiene in enterprise environments. This project marks a solid starting point in my journey toward professional penetration testing and Windows security.

Acknowledgment

I learned and practiced these techniques through TCM Security and Heath Adams (The Cyber Mentor), whose training and YouTube content provided clear, beginner-friendly explanations that helped me understand each attack conceptually and practically.

Thank You

Thank you for taking the time to review my work. This project has been an important step in developing my foundational skills in Active Directory security and ethical hacking. I'm grateful for the learning resources provided by TCM Security and Heath Adams (The Cyber Mentor), whose clear explanations and practical demonstrations made it possible for me to build and understand these AD attack scenarios.

I look forward to continuing my learning journey and exploring more advanced topics in penetration testing and cybersecurity.